

Overview

serdes_top.v is the top-level RTL module for a GateMate FPGA SERDES loopback/PRBS test design. It instantiates the CC_SERDES primitive and wraps it with:

- a 10 MHz system clock heartbeat,
- a button-debounce and start-latch circuit,
- a 3-register FSM for SERDES PLL configuration via the Regfile interface,
- an automatic RX error-recovery watchdog, and
- an 8-LED status display.

The code is well-structured and the intent is clear. However, one critical compilation error and several design-reliability issues must be addressed before the design is production-ready.

Summary of Findings

Priority	Issue	Location
CRITICAL	TX_PAT identifier undeclared -- module will not compile	Line 138
MUST FIX	led[7] never driven -- 8th LED pin is uncontrolled	Lines 207-213
MUST FIX	CDC violation: started/serdes_rst cross clock domains unsafely	Lines 32-33, 96, 187
SHOULD FIX	reg_rdy ignored: FSM does not wait for register-ready handshake	Lines 42, 64-75
MINOR	Timing comment '~300ms' does not match actual cycle count	Line 105
MINOR	err_cnt dual-purpose naming obscures recovery hold-time role	Lines 93-110
MINOR	State 5 is a permanent sink -- re-init requires full board reset	Line 77

Critical Bug

CRITICAL TX_PAT Identifier Undeclared [Line 138]

The localparam that defines TX_PAT was commented out during development (line 91) but the signal name is still passed to the CC_SERDES TX_DATA_I port. Every simulator and synthesiser will reject this as an undeclared identifier, halting the build completely.

```
// localparam [63:0] TX_PAT = 64'hA5A5A5A5_A5A5A5A5; <- commented out
...
.TX_DATA_I (TX_PAT), // <- ERROR: TX_PAT does not exist in this scope
```

Root cause: The constant was probably disabled to switch to a pure PRBS test (TX_PRBS_SEL_I = 3'b001), which causes the hardware to ignore TX_DATA_I entirely. However, RTL must still be syntactically and semantically valid regardless of runtime behaviour.

Important note: When PRBS mode is selected (TX_PRBS_SEL_I != 3'b000), the CC_SERDES block generates its own test pattern internally and the value on TX_DATA_I is not transmitted. The port connection is therefore electrically harmless once fixed, but the signal reference must be valid for the design to compile.

Fix option A (preferred): Uncomment the localparam. This is self-documenting and preserves the design intent should someone switch back to data-pattern mode.

Fix option B: Replace TX_PAT with the literal 64'h0000000000000000, or tie it to rx_data if a loopback data path is intended for future use.

Must-Fix Issues

MUST FIX led[7] Never Driven [Lines 207-213]

The top-level output port is declared output wire [7:0] led, giving eight pins to the LED array. The assign block drives only led[0] through led[6]. led[7] has no driver.

In synthesis this typically results in the pin being tied to a constant 0, meaning the eighth LED on the board is always dark regardless of system state. This wastes a visible status indicator and may mask a latent design issue (perhaps the author intended to display rx_prbs_err or serdes_rst as a second indicator).

Most synthesisers will emit a warning about an undriven output, but because the design still compiles and appears to work on hardware, such warnings are easy to overlook.

```
assign led[0] = ~hb_cnt[23]; // heartbeat
assign led[1] = ~started;
assign led[2] = ~tx_done;
assign led[3] = ~rx_done;
assign led[4] = ~pll_cnt[26]; // SERDES clock alive
assign led[5] = ~loopback_ok;
assign led[6] = serdes_rst;
// led[7] has no assignment
```

Option A: Assign a meaningful signal, for example:

assign led[7] = ~rx_prbs_err; // PRBS error indicator (active-low)

Option B: If the board has only 7 LEDs, narrow the port to [6:0]. This removes the floating pin and eliminates the synthesis warning at its source.

MUST FIX

Clock Domain Crossing (CDC) Violation [Lines 32-33, 96, 187+]

The design contains two asynchronous clock domains:

- clk_10m : 10 MHz system / button / FSM clock
- serdes_clk : high-frequency clock from the CC_SERDES PLL (typically hundreds of MHz)

Two control signals are driven in the clk_10m domain but consumed in the serdes_clk domain with no synchronisation logic between them:

started - registered on clk_10m (line 22), read by the err_cnt watchdog on serdes_clk (line 96). Although the watchdog checks rx_recover and rx_prbs_err rather than started directly, started drives serdes_rst which enters the SERDES block and affects serdes_clk-domain state.

serdes_rst - registered on clk_10m (line 33), used as a reset/enable qualifier throughout the serdes_clk domain.

When a flip-flop samples a signal that was driven by a different clock, the setup and hold times of the sampling flip-flop are not guaranteed. This can cause metastability: the output of the flip-flop enters an intermediate voltage level that eventually resolves to 0 or 1, but the resolution time is unbounded. A metastable value that propagates into downstream combinational logic can corrupt multiple registers in the same clock cycle.

For control signals like serdes_rst and started, a glitch can cause a spurious mid-operation reset of the RX subsystem, triggering a false recovery cycle or permanently asserting serdes_rst in the serdes_clk domain.

```
// CURRENT (unsafe -- no synchroniser):
always @(posedge clk_10m) serdes_rst <= ~started; // clk_10m domain
...
always @(posedge serdes_clk) begin // serdes_clk domain
    if (rx_recover) ... // serdes_rst consumed here without synchronisation

// FIXED -- 2-FF synchroniser for each crossing signal:
```

serdes_top.v -- RTL Code Review

```
reg started_s1, started_s2;
reg srst_s1,    srst_s2;
always @(posedge serdes_clk) begin
    started_s1 <= started;    started_s2 <= started_s1;
    srst_s1    <= serdes_rst; srst_s2    <= srst_s1;
end
// Use started_s2 and srst_s2 everywhere inside the serdes_clk domain.
```

Add a 2-FF synchroniser for every signal crossing from clk_10m into serdes_clk. Because these are slow-changing single-bit control signals (not multi-bit buses), a simple 2-FF chain is the correct and sufficient solution. For multi-bit buses or counters, a Grey-code or handshake scheme would be required, but that case does not arise in this design.

Should-Fix Issues

SHOULD FIX reg_rdy Handshake Ignored in FSM [Lines 42, 64-75]

The CC_SERDES Regfile interface provides a REGFILE_RDY_O ready strobe to signal when a register write has been accepted and the bus is free for the next transaction. This output is wired to reg_rdy (line 42) but the signal is never read by the FSM.

Instead, the FSM waits a fixed 32 cycles in state 3 (wcnt == 6'd31) before issuing the next write. At 10 MHz this is 3.2 us. If the Regfile interface is not ready within that window -- for example at cold start when the PLL has not yet stabilised and internal clocks may be running slowly -- the FSM will begin the next write while the bus is still busy, silently overwriting or losing the in-progress transaction.

The three register writes configure:

Reg 0x51 PLL multiplication ratio and reference divider

Reg 0x5C Global SERDES enable

Reg 0x50 PLL power-on

If any write is silently dropped, the SERDES PLL will not lock. The design will appear to complete initialisation (the FSM reaches state 5 without error) but the link will never come up. This kind of fault is extremely difficult to diagnose without a logic analyser because there is no visible error signal.

```
// CURRENT state 3 -- fixed delay, ignores reg_rdy:
3'd3: begin
    wcnt <= wcnt + 1;
    if (wcnt == 6'd31) fsm <= 3'd4; // proceeds regardless of bus state
end

// IMPROVED -- wait for reg_rdy with timeout watchdog:
3'd3: begin
    wcnt <= wcnt + 1;
    if (reg_rdy || wcnt == 6'd63) begin // normal path or timeout
        wcnt <= 6'd0;
        fsm  <= 3'd4;
    end
end
end
```

Gate the state-3 to state-4 transition on reg_rdy. Retain wcnt as a watchdog timeout (to prevent the FSM from hanging forever if reg_rdy is stuck low due to a hardware fault), but the normal path should use the handshake. Consider asserting an error LED if the timeout fires rather than reg_rdy.

Minor Issues

MINOR Timing Comment Mismatch [Line 105]

The inline comment on line 105 states '~300ms', referring to the PRBS-error duration threshold before the auto-recovery

mechanism fires:

```
if (err_cnt == 25'h1FFFFFF) begin // ~300ms
    rx_recover <= 1'b1;
end
```

25'h1FFFFFF equals 33,554,431 in decimal. For this to equal 300 ms, serdes_clk would need to run at approximately 111.8 MHz. Typical GateMate SERDES configurations run at 500 MHz or higher:

At 500 MHz : $33,554,431 / 500,000,000 = \sim 67 \text{ ms}$

At 1 GHz : $33,554,431 / 1,000,000,000 = \sim 33 \text{ ms}$

Neither figure matches the comment. An incorrect timing comment is actively misleading: a future engineer may 'correct' the threshold to match the 300 ms comment and inadvertently change the recovery behaviour. The recovery hold-time value (25'h20000FF = 33,554,687) has the same problem -- it is only 256 cycles longer than the fault-detection threshold, giving a very short reset pulse whose duration also depends on the undocumented clock rate.

Replace magic numbers with named localparams that encode the frequency assumption:

```
localparam integer SERDES_CLK_HZ = 500_000_000;
localparam integer ERR_HOLD_MS = 100;
localparam integer RECOVER_HOLD_MS = 10;
localparam [24:0] ERR_HOLD_CYC =
    (SERDES_CLK_HZ / 1000) * ERR_HOLD_MS;
localparam [24:0] RECOVER_HOLD_CYC =
    (SERDES_CLK_HZ / 1000) * RECOVER_HOLD_MS;
```

This makes the intended timing explicit and automatically scales if the clock changes.

MINOR

err_cnt Dual-Purpose Naming [Lines 93-110]

The register err_cnt serves two distinct roles within the same always block:

Role 1 (fault detection): In the else-if (rx_prbs_err && rx_done) branch, err_cnt counts consecutive serdes_clk cycles during which the PRBS checker reports an error. When the count reaches the threshold, rx_recover is asserted.

Role 2 (recovery timer): In the if (rx_recover) branch, the same register counts how long the recovery reset has been held asserted. When the count reaches the hold-time value, rx_recover is de-asserted and the counter resets.

The counter is reset at the boundary between the two paths (line 101, line 109) so the arithmetic is correct and the behaviour is as intended. However, reading 'err_cnt <= err_cnt + 1' inside 'if (rx_recover)' is confusing: at that point the counter is acting as a hold-time timer, not an error counter. A reviewer must carefully trace the control flow to understand why an 'error counter' is incrementing during recovery.

```
// Current -- single dual-purpose register:
reg [24:0] err_cnt = 0;

// Suggested -- two named registers:
reg [24:0] fault_dur_cnt = 0; // cycles link has been in error
reg [24:0] recover_hold_cnt = 0; // cycles recovery reset has been asserted

always @(posedge serdes_clk) begin
    if (rx_recover) begin
        recover_hold_cnt <= recover_hold_cnt + 1;
        if (recover_hold_cnt == RECOVER_HOLD_CYC) begin
            rx_recover <= 1'b0;
        end
    end
end
```

```

        recover_hold_cnt <= 0;
    end
end else if (rx_prbs_err && rx_done) begin
    fault_dur_cnt <= fault_dur_cnt + 1;
    if (fault_dur_cnt == ERR_HOLD_CYC)
        rx_recover <= 1'b1;
    end else begin
        fault_dur_cnt <= 0;
    end
end
end

```

Either split into two registers as shown above (clearest approach), or keep a single register but rename it to `timeout_cnt`, which accurately describes its role in both branches. Splitting is preferred because it allows the thresholds to be different sizes if required in future.

MINOR**FSM State 5 is a Permanent Sink [Line 77]**

After completing the three SERDES Regfile writes, the FSM enters state 5 and stays there permanently:

```
3'd5: fsm <= 3'd5; // holds here forever
```

For a one-shot initialisation sequence this is intentional, but it has two practical consequences that are worth acknowledging:

1. Pressing SW3 a second time has no effect on the FSM. The button logic will re-latch `started = 1` (because `btn_cnt` was already saturated from the first press and does not reset), but the FSM in state 5 ignores `started`. The user may be confused when repeated button presses do not cause a SERDES re-initialisation.

2. Recovery from a bad init requires a full power cycle. If the CC_SERDES Regfile writes produce incorrect values (e.g. due to the `reg_rdy` issue above), there is no way to re-run the configuration sequence without power-cycling the board. For a test fixture this is inconvenient; for a deployed system it could mean an unnecessary outage.

The current behaviour is not necessarily wrong -- a one-shot FSM is a legitimate design choice. But it should be a deliberate choice documented in the code, not an unexamined default.

If re-init on button press is desirable, modify state 5 to return to state 0 when the button is released and re-pressed:

```

3'd5: if (!started) begin
    btn_cnt <= 16'd0;
    fsm    <= 3'd0;
end

```

If the one-shot behaviour is intentional, add an explicit comment to state 5:

```
3'd5: fsm <= 3'd5; // one-shot init complete -- power cycle to re-run
```

Strengths

The following aspects of the design are well-executed and worth preserving:

Button debounce quality: The 16-bit counter at 10 MHz requires the button to be held for ~6.5 ms (65,535 cycles) before the latch fires. This is a robust debounce window that filters all ordinary contact bounce and most electrical interference.

Auto-recovery watchdog architecture: The mechanism that detects sustained PRBS errors and triggers a targeted CDR + RX reset -- without disturbing the TX side or requiring any intervention from the FSM -- is a clean, practical approach to link self-healing. The two-threshold design (wait for fault, hold reset, then release) correctly avoids the reset-thrashing that a naive single-threshold design would exhibit.

LED diagnostic coverage: Seven independent status signals are mapped to individual LEDs in a logical order: system heartbeat, start latch, TX ready, RX ready, SERDES clock alive, link OK, and reset state. Together these signals cover every major subsystem, enabling rapid visual triage without a logic analyser or JTAG probe.

FSM clarity: The 6-state register-write sequencer has a single responsibility per state and linear transitions, making the initialisation sequence straightforward to trace by inspection. The separation of address/data setup (state 1), strobe assertion (state 2), and delay (state 3) mirrors the Regfile bus protocol clearly.